# Problems at Speed

On the morning of 1 August 2012, a technician at Knight Capital installed new software on seven of the firm's eight trading servers. Forty-five minutes after the market opened, Knight had lost $460 million.

The eighth server still carried a piece of code called Power Peg, which Knight had stopped using in 2003 but never removed. The new software reused an old flag to switch on a new feature, and on seven servers the flag did what it was meant to do. On the eighth, it woke up Power Peg, which started sending orders and did not stop. According to the Securities and Exchange Commission's order against the firm, Knight's router *"obtained over 4 million executions in 154 stocks for more than 397 million shares"* in those forty-five minutes. By the time it was unwound, the firm was carrying about $3.5 billion of stock it did not want on one side and about $3.15 billion on the other.

Two more details from the same order matter more than the dollar figure. Before the market opened, Knight's system sent 97 automated emails to a group of its own staff warning that something was wrong, and *"these notifications were not acted upon."* And, in the SEC's words, *"Knight did not have written code development and deployment procedures"* for that system, and *"did not require a second technician to review code deployment."*

So, one skipped copy step, on one server out of eight. The defect was tiny. The number of times it ran was not.

Knight was not an AI story. It was a 2012 trading router, and that is exactly why it is worth starting with, because the arithmetic that sank Knight is the arithmetic this whole week kept circling, and it is not new. What is new is how many businesses are about to be running it.

## The recall math

This is the thread that ran under every piece this week, whatever each one was nominally about.

It is tempting to say AI output scales like manufacturing, and in one sense it does: you build the process once and it stamps out units. But a recall does not change the fact that the implication cost is much higher here as a function of the cost per unit. Most of these costs per unit are under $100, but they might execute 100,000 times. (Those two numbers are my illustration, not anybody's dataset. The point is the shape, not the figures.)

Let’s take a minute to work it through, because these are big numbers that aren’t easy enough to see for most of us.  A car maker that finds a fault in a part recalls the cars it shipped, and the number of cars is finite and known. That carmaker additionally has years to correct this problem,

a known protocol for doing so (that National Transportation and Safety Board, I think) <verify>, and has a cost basis, a run rate, and a completion timeline consistent with the recalled item’s place in the manufacturing wrong.

Compare that, then, to a piece of AI-built work, whether that is an agent's configuration, a generated integration, a prompt sitting inside a customer workflow, is not shipped once. It runs. Every run is another unit, and every unit carries whatever was wrong with the first one. So the cost of a defect is not a line item. It is the cost per unit times the run count, and the run count is the number that keeps growing after most people have stopped looking at it.

That is why the falling unit cost this week is not the good news it looks like on its own. Warp's own published benchmark took its internal cost per pull request (a proposed change to the code) from about $80 to about $30 after swapping the model underneath. Nothing about that number is wrong. But a cheaper unit is an invitation to run more units, and every extra run is another copy of anything that is broken.

## When the invoice stops being the signpost

On Thursday I said we would come back to what it means when the invoice stops being the signpost, and this is where it lands.

As long as every model call costs something, the bill is a crude but honest gauge of how often a thing is running. Somebody in finance sees the number go up and asks why. When the model moves onto the desk and the per-call cost goes to zero, that gauge disappears. The work still runs, maybe more than ever, but nothing on an invoice tells you how many times.

Put Thursday next to Monday and you get the uncomfortable version. The unit is getting cheaper, which pushes run count up, and the one instrument most organizations use to notice and track run count is going away at the same time. The multiplier goes up while the number you were watching goes down.

Wednesday showed what that looks like when it goes wrong at the other end of the week. Brevo, an email-marketing company, kept one long-lived cloud key in its source code. Someone found it, and for about four and a half hours on 14 September the forms script, chat widget and loader that Brevo's customers had embedded in their own websites served a fake "verify you are human" page to visitors. One credential, one defect, and it ran on every site that loaded those scripts during those hours. The customers did nothing wrong. They were simply part of the run count.

And Monday's numbers point the same direction, with a caveat I will repeat because it matters. Faros AI, which sells engineering analytics, found across roughly twenty-two thousand developers that at high AI adoption, incidents per pull request were up 242.7% and median time in review was up 441.5%. Those are correlations across teams, not proof that AI caused the

incidents. But they are the direction you would expect if output grew faster than the care going into it, and I have not seen a published dataset that points the other way.

## Craftsmanship, and the loop

There are two different places to put human effort into work that runs at volume, and the industry keeps talking as if they are the same place.

Craftsmanship is at the root of where you get the output without the incidents.

Human-in-the-loop attention is where you get the incidents across the output. The first happens before the run: somebody who knows the work well builds the thing carefully, tests it against the cases that matter, writes down how it gets deployed, and has a second person look before it goes. The second happens after the run has started: somebody watches what comes out, notices when it looks wrong, and stops it. Both are real. They are not interchangeable, and at volume the difference between them is the whole bill.

Knight had both, in a sense. It had the attention: 97 warning emails is a loop, and a fairly loud one. What it did not have was the craft: no written deployment procedure, no second technician, nothing that would have caught a server that never got the new code. The attention arrived after the defect was live, and by then every minute was, on average, close to ninety thousand more executions. *You cannot watch your way out of a problem that is already running a hundred thousand times.* You can only build your way out of it before it starts.

Greg Isenberg (an entrepreneur who runs a small holding company and a startup-ideas podcast) put the shift neatly in a video on Monday: *"The people who used to do the work are becoming the people who check it."* He is describing something real, and in the businesses he is talking about it is probably the right design. But notice where it puts the people. Checking is attention. It sits after the run. And at volume, the checker is not looking at every unit, they are looking at a sample, while the queue grows (Faros's 441.5% is what that queue looks like), and while the vendors selling the tools work to drive the number of human touches per unit down. Warp tracks "human interactions per pull request" and says plainly that the goal is fewer of them. That is a perfectly sensible productivity metric. It is also a count of control points being removed.

None of which makes checking worthless. It makes checking the second line, and it means the first line, the craft, is the one getting thinner just as the run counts go up and the checking goes down.

## Craftsmanship is getting scarce

Here is the part that I think leaders are going to have to own, because almost every other incentive in the organization points the other way.

The essay Friday's piece mentioned, "Software Craftsmanship Is Dead," has been making the rounds since January, and its complaint is about incentives: engineers get rewarded for shipping fast and measurable work, so the person who says *"I need to spend more time to get this right"* loses the argument to the sprint board. That was true before any of this. AI makes it sharper, because now the fast option is not just faster, it is nearly free, and the careful option still costs exactly what it always did. An engineer's afternoon spent getting the deployment procedure right does not get cheaper when the code does.

Oliver Ye, who posts on X as @_isomerism, put it more bluntly ten days ago in a post that went around: *"the moat is literally just caring about your work and being proud of your output and holy shit it does not seem to apply to 99% of people."* Leave the 99% aside, which is a joke, not a census. The first half is the point. Caring about the work is a moat precisely because it is expensive to keep up, and almost everything about the current toolset makes it cheaper not to.

> The messy middle, which is where so many of us have lived for so many years of our career, is shifting, and the implications are massive.

So craftsmanship is becoming scarce, and it is the thing a leader has to emphasize if they want their people to have it and their stuff to work. Not just speed. Speed is the part every tool on the market is already selling you. The part the vendors are not selling is the person who builds it right the first time, because that person does not show up in a productivity chart, and the incidents they prevent do not show up anywhere at all.

Judgment sits inside this rather than on top of it. Knowing which part of a piece of work will run a hundred thousand times, and therefore deserves the extra afternoon, is a judgment call, and it is the call that tells a craftsman where to spend the craft. But it only matters in combination with the rest: the cheap unit, the invisible run count, the thinning first line. On its own it is a slogan.

## The other class of work

On Friday I drew a line between utilities and full solutions, and said Sunday was about the second class, where the argument is a different one. This is the different argument.

A utility (the PDF reader, the summarizer, the small tool that does one job until the platform does it too) can survive being built quickly and roughly, because its life is short and the platform is going to replace it anyway. If it breaks, somebody downloads a different one. The craftsmanship essay is largely right about that layer, and Friday's piece took it seriously for exactly that reason.

A full solution is the system a business actually runs on: the claims workflow, the integration into the finance system, the agent that touches customer records. It does not get replaced by next quarter's release. It runs inside somebody's operation, day after day, which is exactly the condition under which the recall math bites. A rough utility fails once, for one person. A rough

full solution fails every time it runs, for everyone it runs for. That is the class of work where craft stops being a nice-to-have and becomes the main control you have.

## The best case against this

The strongest counter-argument this week came from Ethan Mollick (a Wharton professor, and one of the more careful voices on how organizations actually use AI). In "The Dot and the Swarm," published on Thursday, he concedes that he got agent management wrong. He had assumed companies would need to rebuild management for machines, *"but much of management exists to solve problems agents don't have."* His example is a swarm of agents that sent about 2.7 million messages over 88 hours and organized itself toward a result with thin human direction: people decided where to point it, and the agents did the rest.

If that holds, the case for investing in craft gets a lot weaker. Why spend the afternoon when the machines can coordinate themselves?

I think it holds, as far as coordination goes, and I do not think it touches the recall math. 2.7 million messages is a run count. Whatever was wrong in how the swarm was set up, the prompt, the tools, the access it was given, the definition of done, was present in every one of those messages. The swarm removes the human friction from organizing the work. It does not remove the need for someone to have built the starting conditions well, and it raises the cost of not having done so, because it runs faster and longer than any team of people would. That is Knight's eighth server again, with better manners.

## What to do with this

**Count runs, not invoices.** For anything automated that your business now depends on, find out roughly how many times it runs in a day and who would know if that number doubled. If the honest answer is nobody, you have lost the signpost Thursday described and have not replaced it.

**Spend the craft before the first run.** Review the setup, not just the output: the configuration, the credential it holds, the deployment step, the cases it was tested against. Write the deployment procedure down and make a second person sign it, which is the single control the SEC said Knight did not have. It is cheap, it is boring, and it is the part that prevents rather than catches.

**Protect the person who asks for another afternoon.** When someone on your team says they need more time to get something right, and the thing in question is going to run inside a customer's operation, treat that as the most valuable sentence in the sprint meeting rather than the most annoying one. Measure what they prevent, even roughly, because nothing else in your reporting will.

**Make the alarm somebody's.** Attention still matters; it is just the second line. Knight's 97 emails went to a group. Pick one named person for each system who owns the warning, and make sure they know they own it before it fires.

## Speed is not the virtue

Every leg of this week ended on the same arithmetic. The cost per unit is falling, the count of units is climbing, the instrument that used to show the count is disappearing, and the place where a problem gets caught has moved from before the run to somewhere well after it.

Speed was never the virtue on its own. Problems at speed do nothing except accelerate the total impact of those problems, and the only thing that slows them down is somebody who cared enough to build it right before it ever ran.

---

*Thanks for reading. I know these run long and land dense, and I appreciate you taking the time with them.*

*Lars Klander writes about cybersecurity strategy, AI governance, and technology risk. Today's argument is that cheap AI output turns every defect into its unit cost times its run count, that the attention we put after the run catches problems only once they have multiplied, and that craftsmanship before the run is becoming scarce and is now the thing leaders have to protect.*